

TFCE06_splitPlot.Rmd

Yufang

2026-07-08

```
## =====
## TFCE analysis in R for exported split-plot EEG CSV
##
## Input files:
##   ../data/06_designTable.csv
##   ../data/06_EEGdata_long.csv
##
## Model:
##   Amplitude ~ GroupCode * CondCode + (1|Subject)
##
## Tests:
##   1. Group main effect
##   2. Condition main effect
##   3. Group x Condition interaction
##
## Permutation:
##   Group effect:
##     Shuffle GroupCode between subjects, keeping both rows together
##
##   Condition effect:
##     Rearrange CondCode within each subject
##
##   Interaction effect:
##     Here follows the same simplified synchronized permutation as CondCode.
##     If using Freedman--Lane for interaction, replace this part accordingly.
##
## EEG amplitudes are never directly permuted.
## =====

rm(list = ls())

library(dplyr)

##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##   filter, lag

## The following objects are masked from 'package:base':
```

```
##
## intersect, setdiff, setequal, union
```

```
library(tidyr)
library(lme4)
```

```
## Loading required package: Matrix
```

```
##
## Attaching package: 'Matrix'
```

```
## The following objects are masked from 'package:tidyr':
##
## expand, pack, unpack
```

```
library(lmerTest)
```

```
##
## Attaching package: 'lmerTest'
```

```
## The following object is masked from 'package:lme4':
##
## lmer
```

```
## The following object is masked from 'package:stats':
##
## step
```

```
source("../R/tfce_transform.R")
cat("\nStarting TFCE split-plot analysis...\n")
```

```
##
## Starting TFCE split-plot analysis...
```

```
## =====
## -1. Load CSV files
## =====
```

```
designTable <- read.csv("../data/06_designTable.csv")
EEGLong <- read.csv("../data/06_EEGdata_long.csv")

print(names(designTable))
```

```
## [1] "Subject" "GroupCode" "GroupName" "CondCode" "CondName"
```

```
print(names(EEGLong))
```

```
## [1] "Subject" "GroupCode" "CondCode" "Channel" "Time" "Amplitude"
```

```

## =====
## 0. Merge EEG data with design table
## =====

designTable <- designTable %>%
  mutate(RowID = row_number())

dat <- EEGlong %>%
  left_join(
    designTable,
    by = c("Subject", "GroupCode", "CondCode")
  ) %>%
  mutate(
    Subject = factor(Subject),
    Channel = factor(Channel),
    GroupCode = as.numeric(GroupCode),
    CondCode = as.numeric(CondCode),
    Interaction = GroupCode * CondCode
  )

channels <- unique(dat$Channel)
times <- sort(unique(dat$Time))

n_chan <- length(channels)
n_time <- length(times)

## =====
## Basic checks
## =====

subjects <- unique(designTable$Subject)

for (s in subjects) {

  tmp <- designTable %>%
    filter(Subject == s)

  if (nrow(tmp) != 2) {
    stop("Each subject must have exactly two rows.")
  }

  if (!all(sort(tmp$CondCode) == c(-1, 1))) {
    stop("Each subject must have one Noun (-1) and one Verb (1).")
  }

  if (length(unique(tmp$GroupCode)) != 1) {
    stop("GroupCode must be constant within each subject.")
  }
}

## =====
## -2. Function to compute LME t-maps
## =====

```

```

compute_tmaps <- function(data) {
  res <- data %>%
    group_by(Channel, Time) %>%
    group_modify( ~ {
      fit <- lmer(Amplitude ~ GroupCode + CondCode + (1 | Subject),
                  data = .x)

      cf <- coef(summary(fit))

      tibble(t_Group = cf["GroupCode", "t value"], t_Cond = cf["CondCode", "t value"], )
    }) %>%
    ungroup()

  make_mat <- function(x) {
    tmat <- res %>%
      mutate(
        Channel = factor(Channel, levels = channels),
        Time = factor(Time, levels = times)
      ) %>%
      arrange(Channel, Time) %>%
      pull(x)

    matrix(
      tmat,
      nrow = n_chan,
      ncol = n_time,
      byrow = TRUE,
      dimnames = list(channels, times)
    )
  }

  list(Group = make_mat("t_Group"), Cond = make_mat("t_Cond"))
}

## =====
## Step 1: Observed t-maps
## =====

cat("Step 1: Computing observed t-maps...\n")

```

```
## Step 1: Computing observed t-maps...
```

```

t_Obs <- compute_tmaps(dat)

t_Obs_Group <- t_Obs$Group
t_Obs_Cond <- t_Obs$Cond

cat("Observed t-maps completed.\n")

```

```
## Observed t-maps completed.
```

```
## =====
## Step 2: Observed TFCE maps
## =====

cat("Step 2: Computing observed TFCE maps...\n")

## Step 2: Computing observed TFCE maps...

TFCE_Obs_Group <- tfce_transform(t_Obs_Group, E = 0.66, H = 2, dh = 0.1)
TFCE_Obs_Cond  <- tfce_transform(t_Obs_Cond,  E = 0.66, H = 2, dh = 0.1)

cat("Observed TFCE maps completed.\n")
```

```
## Observed TFCE maps completed.
```

```
## =====
## Step 3: Permutation distribution
## =====

set.seed(123)

n_perm <- 1

TFCE_permMax_Group <- rep(NA, n_perm)
TFCE_permMax_Cond  <- rep(NA, n_perm)

subject_info <- designTable %>%
  select(RowID, Subject, GroupCode, CondCode)

subj_group <- subject_info %>%
  group_by(Subject) %>%
  summarise(GroupCode = unique(GroupCode), .groups = "drop")

for (p in seq_len(n_perm)) {

  ## -----
  ## 3.1 Permute GroupCode between subjects
  ## -----

  subj_group_perm <- subj_group %>%
    mutate(GroupCode_perm = sample(GroupCode))

  perm_info_group <- subject_info %>%
    select(-GroupCode) %>%
    left_join(
      subj_group_perm %>% select(Subject, GroupCode_perm),
      by = "Subject"
    ) %>%
    rename(GroupCode = GroupCode_perm)

  perm_dat_group <- EEGlong %>%
```

```

select(Subject, CondCode, Channel, Time, Amplitude) %>%
left_join(
  perm_info_group,
  by = c("Subject", "CondCode")
) %>%
mutate(
  Subject = factor(Subject),
  Channel = factor(Channel),
  GroupCode = as.numeric(GroupCode),
  CondCode = as.numeric(CondCode),
  Interaction = GroupCode * CondCode
)

## -----
## 3.2 Permute CondCode within subjects
## -----

perm_info_cond <- subject_info %>%
  group_by(Subject) %>%
  mutate(CondCode = sample(CondCode)) %>%
  ungroup()

perm_dat_cond <- EEGlong %>%
  select(Subject, CondCode, Channel, Time, Amplitude) %>%
  left_join(
    perm_info_cond,
    by = c("Subject", "CondCode")
  ) %>%
  mutate(
    Subject = factor(Subject),
    Channel = factor(Channel),
    GroupCode = as.numeric(GroupCode),
    CondCode = as.numeric(CondCode),
    Interaction = GroupCode * CondCode
  )

## -----
## 3.3 Compute permutation t-maps
## -----

t_perm_group <- compute_tmaps(perm_dat_group)
t_perm_cond <- compute_tmaps(perm_dat_cond)

TFCE_perm_Group <- tfce_transform(
  t_perm_group$Group,
  E = 0.66,
  H = 2,
  dh = 0.1
)

TFCE_perm_Cond <- tfce_transform(
  t_perm_cond$Cond,
  E = 0.66,

```

```

    H = 2,
    dh = 0.1
  )

  TFCE_permMax_Group[p] <- max(abs(TFCE_perm_Group), na.rm = TRUE)
  TFCE_permMax_Cond[p] <- max(abs(TFCE_perm_Cond), na.rm = TRUE)

  if (p %% 50 == 0) {
    cat("Permutation", p, "of", n_perm, "\n")
  }
}

cat("Permutation testing completed.\n")

```

```
## Permutation testing completed.
```

```

## =====
## Step 4: TFCE-corrected significance
## =====

cat("Step 4: Computing TFCE-corrected significance...\n")

```

```
## Step 4: Computing TFCE-corrected significance...
```

```

alpha <- 0.05
nPerm <- length(TFCE_permMax_Group)

maxTFCE_Group <- sort(c(
  TFCE_permMax_Group,
  max(abs(TFCE_Obs_Group), na.rm = TRUE)
))

maxTFCE_Cond <- sort(c(
  TFCE_permMax_Cond,
  max(abs(TFCE_Obs_Cond), na.rm = TRUE)
))

maxTFCEcrit_Group <- maxTFCE_Group[round(nPerm * (1 - alpha))]
maxTFCEcrit_Cond <- maxTFCE_Cond[round(nPerm * (1 - alpha))]

Mask_Group <- abs(TFCE_Obs_Group) >= maxTFCEcrit_Group
Mask_Cond <- abs(TFCE_Obs_Cond) >= maxTFCEcrit_Cond

compute_p_values <- function(TFCE_Obs, TFCE_permMax) {

  P <- matrix(
    NA_real_,
    nrow = nrow(TFCE_Obs),

```

```

    ncol = ncol(TFCE_Obs),
    dimnames = dimnames(TFCE_Obs)
  )

  for (ch in seq_len(nrow(TFCE_Obs))) {
    for (tp in seq_len(ncol(TFCE_Obs))) {
      P[ch, tp] <-
        (sum(TFCE_permMax >= abs(TFCE_Obs[ch, tp]), na.rm = TRUE) + 1) /
        (length(TFCE_permMax) + 1)
    }
  }

  P
}

P_Values_Group <- compute_p_values(TFCE_Obs_Group, TFCE_permMax_Group)
P_Values_Cond <- compute_p_values(TFCE_Obs_Cond, TFCE_permMax_Cond)

cat("TFCE-corrected significance completed.\n")

```

```
## TFCE-corrected significance completed.
```

```
cat(sprintf("Critical TFCE Group = %.4f\n", maxTFCEcrit_Group))
```

```
## Critical TFCE Group = 1.9325
```

```
cat(sprintf("Critical TFCE Cond = %.4f\n", maxTFCEcrit_Cond))
```

```
## Critical TFCE Cond = 167.2128
```

```

## =====
## Step 5: Store results
## =====

cat("Step 5: Storing results...\n")

```

```
## Step 5: Storing results...
```

```

Results <- list(

  Obs_Group = t_Obs_Group,
  TFCE_Obs_Group = TFCE_Obs_Group,
  TFCE_Null_Group = TFCE_permMax_Group,
  maxTFCEcrit_Group = maxTFCEcrit_Group,
  P_Values_Group = P_Values_Group,
  Mask_Group = Mask_Group,

  Obs_Cond = t_Obs_Cond,
  TFCE_Obs_Cond = TFCE_Obs_Cond,
  TFCE_Null_Cond = TFCE_permMax_Cond,

```



```

maxTFCEcrit_Cond = maxTFCEcrit_Cond,
P_Values_Cond = P_Values_Cond,
Mask_Cond = Mask_Cond,

alpha = alpha,
nPerm = nPerm,
model = "Amplitude ~ GroupCode * CondCode + (1|Subject)",
times = times,
channels = channels
)

cat("Results stored.\n")

```

Results stored.

```

## =====
## Step 6: Plot function
## =====

plot_tmap <- function(tmap, mask, main_title) {

  sigT <- tmap
  sigT[!mask] <- 0

  times_num <- as.numeric(colnames(tmap))
  channels <- rownames(tmap)
  nChan <- nrow(tmap)

  image(
    x = times_num,
    y = seq_len(nChan),
    z = t(sigT),
    xlab = "Time (ms)",
    ylab = "Channel",
    main = main_title,
    axes = FALSE
  )

  axis(1, at = seq(-200, 800, by = 200))
  axis(
    2,
    at = seq_len(nChan),
    labels = channels,
    las = 2,
    cex.axis = 0.7
  )
  box()
}

plot_tfce_map <- function(tfce_map, main_title) {

  times_num <- as.numeric(colnames(tfce_map))
  channels <- rownames(tfce_map)

```

```

nChan      <- nrow(tfce_map)

image(
  x = times_num,
  y = seq_len(nChan),
  z = t(tfce_map),
  xlab = "Time (ms)",
  ylab = "Channel",
  main = main_title,
  axes = FALSE
)

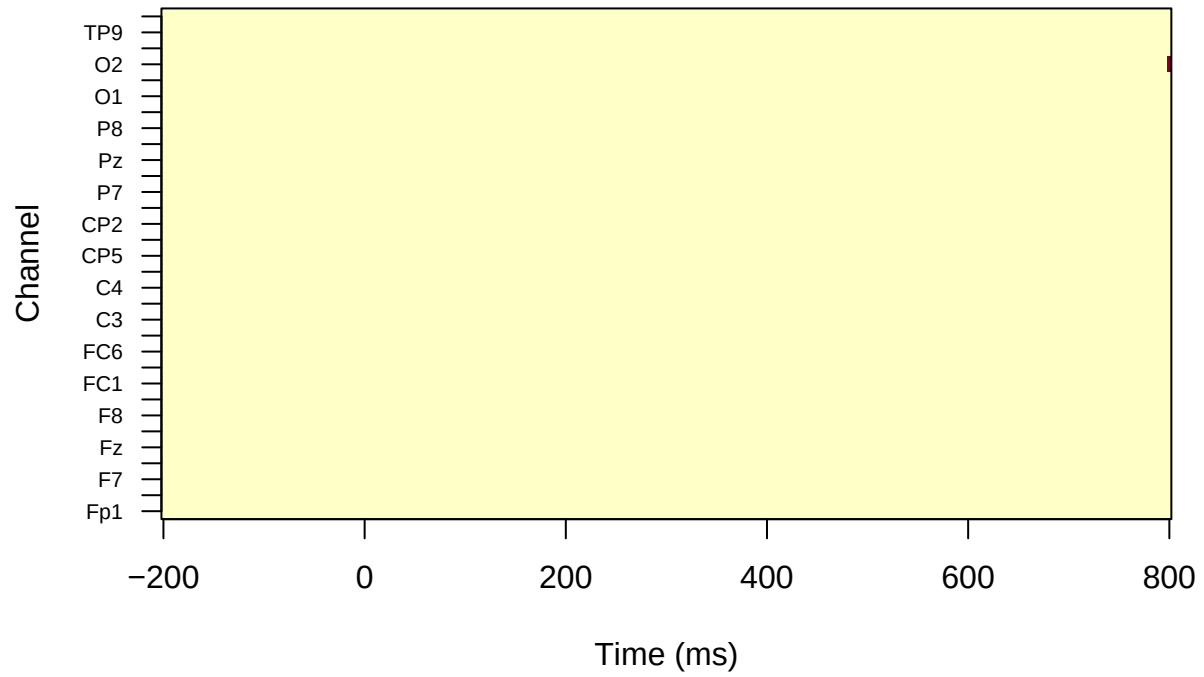
axis(1, at = seq(-200, 800, by = 200))
axis(
  2,
  at = seq_len(nChan),
  labels = channels,
  las = 2,
  cex.axis = 0.7
)
box()
}

## =====
## Step 7: Plot TFCE-corrected significant t-values
## =====

plot_tmap(
  t_Obs_Group,
  Mask_Group,
  "TFCE-corrected Group Effect"
)

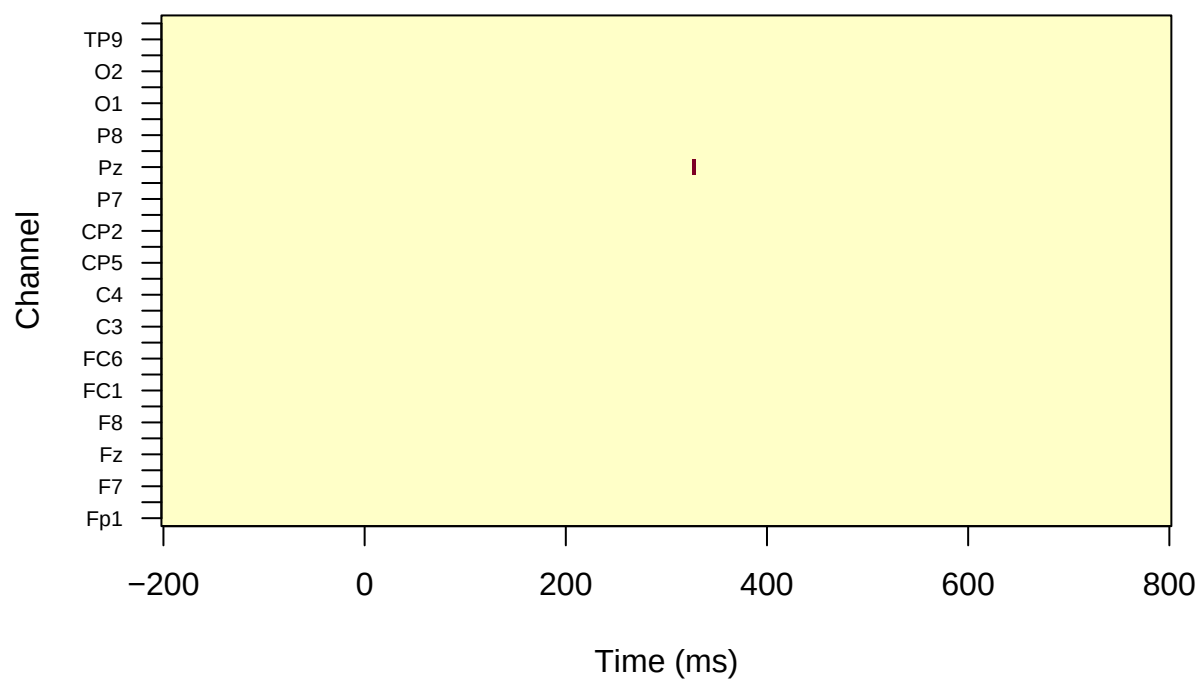
```

TFCE-corrected Group Effect



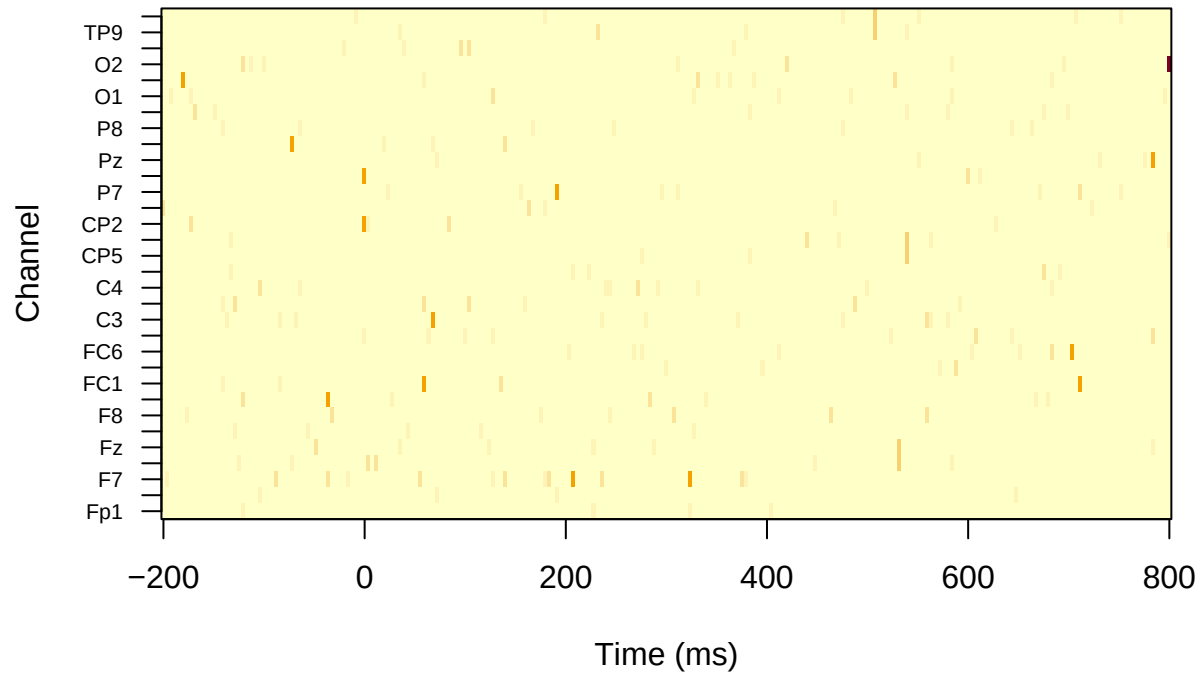
```
plot_tmap(  
  t_Obs_Cond,  
  Mask_Cond,  
  "TFCE-corrected Condition Effect"  
)
```

TFCE-corrected Condition Effect



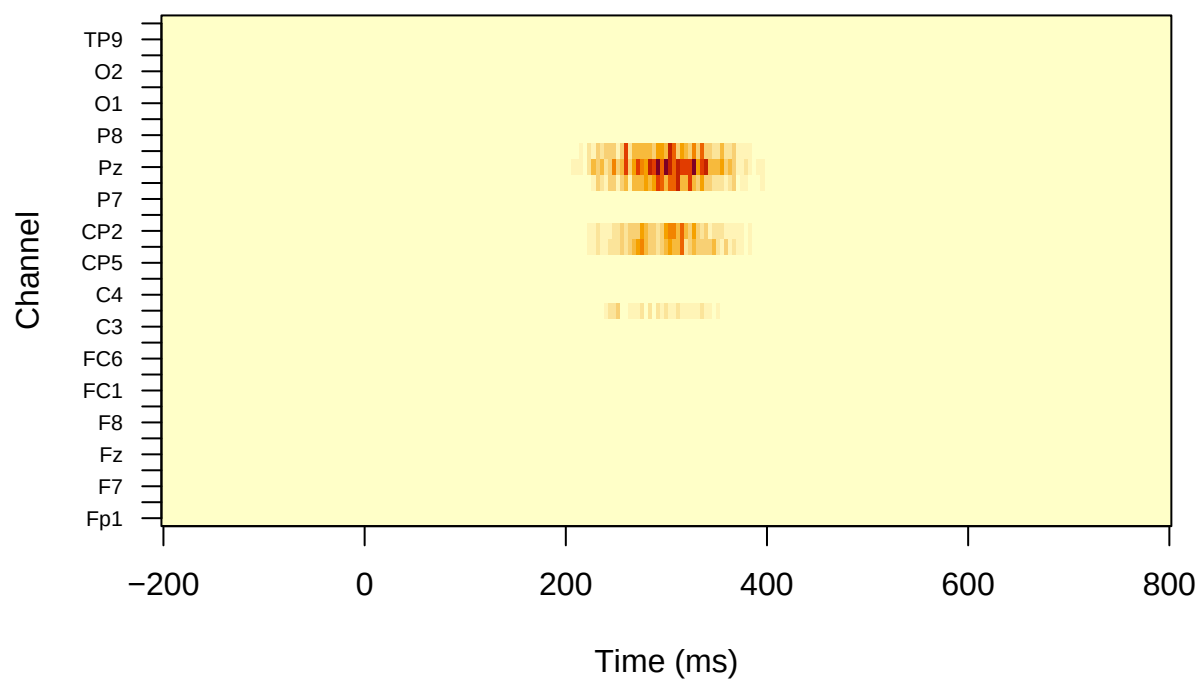
```
## =====  
## Step 8: Plot observed TFCE maps  
## =====  
  
plot_tfce_map(  
  TFCE_Obs_Group,  
  "Observed TFCE Map: Group Effect"  
)
```

Observed TFCE Map: Group Effect



```
plot_tfce_map(  
  TFCE_Obs_Cond,  
  "Observed TFCE Map: Condition Effect"  
)
```

Observed TFCE Map: Condition Effect



```
## =====
## Step 9: Save TFCE results
## =====

if (!dir.exists("../results")) {
  dir.create("../results")
}

save(
  Results,
  t_Obs_Group,
  t_Obs_Cond,

  TFCE_Obs_Group,
  TFCE_Obs_Cond,

  TFCE_permMax_Group,
  TFCE_permMax_Cond,

  maxTFCEcrit_Group,
  maxTFCEcrit_Cond,

  P_Values_Group,
  P_Values_Cond,

  Mask_Group,
```

```
Mask_Cond,  
  
times,  
channels,  
file = "../results/06_TFCE_split_plot_results.RData"  
)  
  
cat("TFCE split-plot analysis completed and results saved.\n")
```

```
## TFCE split-plot analysis completed and results saved.
```